

Synergy Network Enhanced Wallet Security Specification

Version: 2.0

Date: September 2025

Classification: Technical Specification

Abstract

This document specifies a multi-layered security architecture for the Synergy Network wallet that eliminates the traditional single-point-of-failure inherent in seed phrase-only recovery systems. The architecture combines BIP-39 passphrases, device-bound key derivation, post-quantum cryptography, and graduated security levels to create a system where compromise of any single component is insufficient for wallet access.

The system achieves security equivalent to multi-factor authentication while maintaining compatibility with standard recovery procedures through carefully designed fallback mechanisms.

1. Introduction

1.1 Problem Statement

Traditional cryptocurrency wallets rely on a 12 or 24-word seed phrase as the sole recovery mechanism. This creates several critical vulnerabilities:

- **Single Point of Failure:** Anyone obtaining the seed phrase can immediately access all funds
- **Physical Security Risk:** Written seed phrases are vulnerable to theft, loss, or discovery
- **Social Engineering:** Users can be tricked into revealing their seed phrase
- **Quantum Vulnerability:** Current elliptic curve cryptography will be broken by quantum computers

1.2 Design Goals

1. **Eliminate Seed Phrase Sufficiency:** Make possession of the seed phrase alone insufficient for wallet access
2. **Quantum Resistance:** Integrate post-quantum cryptographic algorithms
3. **Graduated Security:** Provide multiple security levels based on user risk tolerance
4. **Recovery Assurance:** Ensure legitimate users can always recover their wallets

- 5. **Standards Compatibility:** Maintain compatibility with BIP-39 and BIP-44 standards
- 6. **Cross-Platform Support:** Work across web, mobile, and desktop environments

1.3 Threat Model

Protected Against:

- Physical theft of written seed phrases
- Digital compromise of stored mnemonics
- Social engineering attacks targeting seed phrases
- Quantum computer attacks on cryptographic keys
- Insider attacks with partial information access
- Brute force attacks on encrypted wallet data

Not Protected Against:

- Compromise of all security factors simultaneously
- Physical coercion with full knowledge of security setup
- Quantum attacks on the underlying blockchain consensus

2. Architecture Overview

2.1 Security Factors

The system implements a three-factor security model:

1. **Knowledge Factor (K):** BIP-39 passphrase - something the user knows
2. **Possession Factor (P):** Device-bound derivation - something the user has
3. **Cryptographic Factor (C):** Enhanced key derivation with PQC - mathematical protection

2.2 Security Levels

Level	K	P	C	Use Case
Basic	✓	-	✓	Standard users, simple backup
Enhanced	✓	✓	✓	Security-conscious users
Maximum	✓	✓	✓+	High-value accounts, institutional use

2.3 Key Derivation Flow

Mnemonic (12/24 words) + Passphrase (K)

↓ BIP-39

Base Seed (512 bits)

↓ + Device Secret (P)

↓ HKDF-SHA256

Enhanced Seed (512 bits)

↓ Domain Separation

PQC Seed (256 bits) + Chain Seeds (256 bits each)

↓

Dilithium Keypair + BIP-44 Chain Keys

3. Cryptographic Specifications

3.1 Hash Functions

- **Primary:** SHA-256 (FIPS 180-4)
- **Key Derivation:** HKDF-SHA256 (RFC 5869)
- **Password Hashing:** PBKDF2-SHA512 with minimum 300,000 iterations

3.2 Symmetric Encryption

- **Algorithm:** AES-256-GCM (FIPS 197)
- **Key Size:** 256 bits
- **IV Size:** 96 bits (randomly generated per encryption)
- **Authentication:** Integrated in GCM mode

3.3 Post-Quantum Cryptography

Primary Algorithm: CRYSTALS-Dilithium

Specification: FIPS 204 (Draft)

- **Security Level:** Dilithium3 (NIST Level 3, equivalent to AES-192)
- **Public Key Size:** 1,952 bytes
- **Private Key Size:** 4,000 bytes
- **Signature Size:** 3,293 bytes

Key Generation:

```
seed ← Enhanced_Seed[0:32] // First 32 bytes as PQC seed  
(pk, sk) ← Dilithium3.KeyGen(seed)
```

Alternative Algorithm: SPHINCS+

Specification: FIPS 205 (Draft)

- **Parameter Set:** SPHINCS+-SHA2-192s-simple
- **Public Key Size:** 48 bytes
- **Private Key Size:** 96 bytes
- **Signature Size:** 16,224 bytes

Used as fallback if Dilithium is compromised or unavailable.

3.4 Key Derivation Function (KDF)

HKDF-SHA256 Parameters

```
IKM = Base_Seed (512 bits)  
Salt = Device_Secret (256 bits)  
Info = "synergy-wallet-v2-" || Security_Level || "\x00"  
Length = 64 bytes (512 bits)  
  
Enhanced_Seed = HKDF-Expand(HKDF-Extract(Salt, IKM), Info, Length)
```

Domain Separation

```
PQC_Seed = Enhanced_Seed[0:32]  
BTC_Seed = Enhanced_Seed[32:40]  
ETH_Seed = Enhanced_Seed[40:48]  
SOL_Seed = Enhanced_Seed[48:56]  
Reserved = Enhanced_Seed[56:64]
```

4. Component Specifications

4.1 BIP-39 Passphrase (Knowledge Factor)

Requirements

- **Minimum Length:** 8 characters
- **Recommended Length:** 20+ characters or 4+ words
- **Character Set:** UTF-8 (full Unicode support)
- **Normalization:** NFKD (as per BIP-39)

Security Properties

- Changes the effective seed without invalidating the mnemonic
- Should be memorizable or stored separately from seed phrase
- Case-sensitive and whitespace-sensitive

Implementation

```
javascript
```

```
const baseSeed = bip39.mnemonicToSeedSync(mnemonic, passphrase);
```

4.2 Device Secret (Possession Factor)

Generation Algorithm

```
javascript
```

```

async function generateDeviceSecret() {
  const timestamp = Date.now();
  const components = [
    navigator.userAgent,
    navigator.language,
    navigator.platform,
    screen.width + 'x' + screen.height + 'x' + screen.colorDepth,
    Intl.DateTimeFormat().resolvedOptions().timeZone,
    navigator.hardwareConcurrency || 0,
    navigator.maxTouchPoints || 0
  ];

  const fingerprint = components.join('|') + '|' + timestamp;
  const deviceBytes = await sha256(fingerprint);

  return {
    deviceSecret: deviceBytes,
    deviceId: toHex(deviceBytes.slice(0, 16)),
    timestamp: timestamp
  };
}

```

Persistence Strategy

Creation Mode:

- Generate new device secret
- Store device ID in wallet metadata
- Device secret used immediately for key derivation

Import Mode:

- Check wallet metadata for existing device binding
- If device-bound: generate new device secret and initiate recovery process
- If not device-bound: proceed with passphrase-only derivation

Recovery Mechanism

When importing a device-bound wallet on a new device:

1. **Immediate:** Allow import with passphrase + recovery email verification

2. **Time-locked:** 24-hour delay for large transactions
3. **Social Recovery:** Guardian confirmation for high-value wallets

4.3 Enhanced Key Derivation

HKDF Implementation

```
javascript

async function deriveEnhancedSeed(baseSeed, deviceSecret, securityLevel) {
  const info = new TextEncoder().encode(`synergy-wallet-v2-${securityLevel}\x00`);

  // HKDF-Extract
  const key = await crypto.subtle.importKey(
    'raw', deviceSecret, 'HKDF', false, ['deriveBits']
  );

  // HKDF-Expand
  const enhancedSeed = await crypto.subtle.deriveBits(
    {
      name: 'HKDF',
      hash: 'SHA-256',
      salt: deviceSecret,
      info: info
    },
    await crypto.subtle.importKey('raw', baseSeed, 'HKDF', false, ['deriveBits']),
    512 // 64 bytes
  );

  return new Uint8Array(enhancedSeed);
}
```

4.4 Wallet Data Encryption

Encryption Process

```
javascript
```

```

async function encryptWalletData(password, data) {
  // Generate salt
  const salt = crypto.getRandomValues(new Uint8Array(32));

  // Derive key using PBKDF2
  const key = await deriveKeyPBKDF2(password, salt, 300000);

  // Generate IV
  const iv = crypto.getRandomValues(new Uint8Array(12));

  // Encrypt using AES-GCM
  const cryptoKey = await crypto.subtle.importKey('raw', key, 'AES-GCM', false, ['encrypt']);
  const ciphertext = await crypto.subtle.encrypt(
    { name: 'AES-GCM', iv: iv },
    cryptoKey,
    new TextEncoder().encode(data)
  );

  return {
    salt: toHex(salt),
    iv: toHex(iv),
    ciphertext: toHex(new Uint8Array(ciphertext))
  };
}

```

5. Wallet Creation Process

5.1 User Flow

1. **Mode Selection:** Create new or import existing wallet
2. **Seed Generation:** Generate 12 or 24-word BIP-39 mnemonic
3. **Seed Verification:** User confirms seed phrase backup
4. **Passphrase Setup:** Mandatory passphrase creation with confirmation
5. **Password Setup:** Local device encryption password
6. **Security Level:** User selects Basic/Enhanced/Maximum
7. **Device Binding:** Generate device secret (Enhanced/Maximum only)
8. **Backup Planning:** Recovery strategy education
9. **Finalization:** Create wallet with all security factors

5.2 Key Generation Process

javascript

```
async function createWallet(mnemonic, passphrase, password, securityLevel) {  
  // 1. Generate base seed  
  const baseSeed = bip39.mnemonicToSeedSync(mnemonic, passphrase);  
  
  // 2. Generate device secret (if applicable)  
  let deviceSecret;  
  if (securityLevel !== 'basic') {  
    const deviceInfo = await generateDeviceSecret();  
    deviceSecret = deviceInfo.deviceSecret;  
  } else {  
    deviceSecret = new Uint8Array(32); // Zero salt for basic mode  
  }  
  
  // 3. Derive enhanced seed  
  const enhancedSeed = await deriveEnhancedSeed(baseSeed, deviceSecret, securityLevel);  
  
  // 4. Generate PQC keys  
  const pqcSeed = enhancedSeed.slice(0, 32);  
  const { publicKey, privateKey } = await generateDilithiumKeypair(pqcSeed);  
  
  // 5. Generate blockchain addresses  
  const btcSeed = enhancedSeed.slice(32, 40);  
  const ethSeed = enhancedSeed.slice(40, 48);  
  const solSeed = enhancedSeed.slice(48, 56);  
  
  const addresses = {  
    bitcoin: generateBTCAddress(mnemonic, passphrase),  
    ethereum: generateETHAddress(mnemonic, passphrase),  
    solana: generateSOLAddress(mnemonic, passphrase),  
    synergy: await pubkeyToSynergyAddress(publicKey)  
  };  
  
  // 6. Encrypt sensitive data  
  const encryptedMnemonic = await encryptWalletData(password, mnemonic);  
  const encryptedPrivateKey = await encryptWalletData(password, toHex(privateKey));  
  const encryptedPassphrase = await encryptWalletData(password, passphrase);  
  
  // 7. Create wallet object  
  return {  
    id: generateWalletId(),  
    version: "2.0",  
  }  
}
```

```

securityLevel: securityLevel,
deviceId: deviceInfo?.deviceId || null,
publicKey: toHex(publicKey),
addresses: addresses,
encryptedData: {
  mnemonic: encryptedMnemonic,
  privateKey: encryptedPrivateKey,
  passphrase: encryptedPassphrase
},
metadata: {
  createdAt: new Date().toISOString(),
  kdfIterations: 300000,
  algorithms: {
    pqc: "Dilithium3",
    kdf: "PBKDF2-SHA512",
    encryption: "AES-256-GCM"
  }
}
};
}

```

6. Security Analysis

6.1 Attack Resistance

Seed Phrase Theft

- **Basic Level:** Attacker needs seed phrase + passphrase
- **Enhanced Level:** Attacker needs seed phrase + passphrase + device access
- **Maximum Level:** Attacker needs seed phrase + passphrase + device access + additional factors

Device Compromise

- Encrypted data requires password + (passphrase for verification)
- Device fingerprinting changes require re-authentication
- Time-locked recovery prevents immediate access

Password/Passphrase Attacks

- PBKDF2 with 300,000 iterations resists brute force

- Passphrase stored encrypted, never in plaintext
- Failed attempts can trigger additional security measures

6.2 Quantum Resistance

Current Implementation

- Dilithium3 provides 192-bit quantum security (NIST Level 3)
- Resistant to known quantum algorithms (Shor's, Grover's)
- Signed transactions remain secure post-quantum

Upgrade Path

- Algorithm agility allows migration to newer PQC algorithms
- Hybrid classical+quantum signatures during transition period
- Backward compatibility with classical addresses maintained

6.3 Recovery Scenarios

Scenario	Basic	Enhanced	Maximum
Lost device	Seed + Passphrase	Seed + Passphrase + Time delay	Seed + Passphrase + Guardian approval
Forgotten passphrase	Recovery impossible	Recovery impossible	Social recovery possible
Lost seed phrase	Recovery impossible	Recovery impossible	Social recovery possible
Device theft	Password protects	Password + device binding protect	Multi-factor protection

7. Implementation Guidelines

7.1 Security Requirements

Cryptographic Libraries

- **Dilithium**: Use NIST-approved implementation
- **AES**: Hardware-accelerated when available
- **HKDF**: Constant-time implementation

- **Random Number Generation:** Cryptographically secure (crypto.getRandomValues)

Memory Management

- Clear sensitive data after use
- Avoid string concatenation for secrets
- Use Uint8Array for binary data
- Implement secure memory zeroing where possible

Key Storage

- Never store plaintext seeds or passphrases
- Use secure enclaves when available (iOS Keychain, Android Keystore)
- Implement proper key lifecycle management

7.2 User Experience Guidelines

Onboarding

- Progressive disclosure of security features
- Clear explanation of each security factor
- Mandatory backup verification
- Recovery testing before wallet activation

Error Handling

- Generic error messages to prevent information leakage
- Rate limiting for authentication attempts
- Clear recovery instructions
- Emergency contact information

7.3 Testing Requirements

Unit Tests

- Cryptographic function correctness
- Key derivation determinism
- Encryption/decryption round-trips

- Error condition handling

Integration Tests

- End-to-end wallet creation
- Recovery procedures
- Cross-platform compatibility
- Performance benchmarks

Security Tests

- Penetration testing
- Side-channel analysis
- Quantum algorithm simulation
- Social engineering resistance

8. Compliance and Standards

8.1 Cryptographic Standards Compliance

- **FIPS 140-2**: Cryptographic modules
- **NIST SP 800-90A**: Random number generation
- **NIST SP 800-108**: Key derivation functions
- **RFC 5869**: HKDF specification
- **FIPS 204**: CRYSTALS-Dilithium (draft)

8.2 Security Frameworks

- **OWASP**: Web application security guidelines
- **NIST Cybersecurity Framework**: Risk management
- **ISO 27001**: Information security management

9. Migration and Compatibility

9.1 Legacy Wallet Support

Existing wallets created with standard seed-phrase-only security can be upgraded:

1. **In-Place Upgrade**: Add passphrase and device binding to existing wallet

2. **Migration:** Create new enhanced wallet and transfer assets
3. **Hybrid Mode:** Maintain both old and new security models during transition

9.2 Cross-Platform Compatibility

- **Web:** Full compatibility with modern browsers
- **Mobile:** Native implementations with secure hardware integration
- **Desktop:** Enhanced security with TPM/Secure Enclave support
- **Hardware:** Integration with hardware security modules

10. Future Enhancements

10.1 Advanced Features

- **Biometric Authentication:** Integration with platform biometric APIs
- **Social Recovery:** Shamir secret sharing with trusted guardians
- **Transaction Policies:** Smart contract-based spending rules
- **Multi-Signature:** M-of-N signature schemes for institutional use

10.2 Research Areas

- **Homomorphic Encryption:** Privacy-preserving computations
- **Zero-Knowledge Proofs:** Private authentication mechanisms
- **Threshold Cryptography:** Distributed key management
- **Hardware Security Modules:** Enhanced tamper resistance

11. Conclusion

The Synergy Network Enhanced Wallet Security Specification provides a comprehensive solution to the fundamental security weaknesses of traditional cryptocurrency wallets. By implementing multi-factor security with post-quantum cryptography, the system achieves security properties previously available only to institutional users while maintaining usability for individual users.

The graduated security model allows users to select appropriate protection levels based on their risk tolerance and technical sophistication. The inclusion of robust recovery mechanisms ensures that enhanced security does not come at the cost of accessibility.

This specification serves as the foundation for implementing next-generation cryptocurrency wallet security that will remain effective in the post-quantum era.

Document History:

- v1.0: Initial draft specification
- v2.0: Added post-quantum cryptography and device binding
- v2.1: Enhanced recovery mechanisms and security analysis

Authors: Synergy Network Security Team

Review Status: Technical Review Complete

Implementation Status: Ready for Development